

Assignment - 02

Name: Bhavik Pahuja

Section: 24AIT_KRG-1_B

UID: 24BAI70522

Semester: 5th

Branch: BE-CSE(AIML)

Subject: Advanced Database
Management System

Question 1:

What is an IP Address? Why do we need classes in case of IPV4 address? Explain each class, Mention how many host and networks are possible in each class. Also explain what is subnetting and supernetting with examples.

Solution:

1. What is an IP Address?

An **IP (Internet Protocol) address** is a logical address assigned to a device/interface on a network. It is used to **identify the source and destination** of packets and enables devices to communicate over an IP network.

For **IPv4**, an IP address is **32 bits** long.

It is divided into **4 octets**, each containing 8 bits:

32 bits

↓

11000000.10101000.00000001.00000001

↓ ↓ ↓ ↓

192 168 1 1

So:

Binary: 11000000.10101000.00000001.00000001

Decimal: 192.168.1.1

An IPv4 address traditionally consists of:

Network ID + Host ID

- **Network ID:** Identifies the network.
- **Host ID:** Identifies a particular device within that network.

2. Why do we need classes in IPv4?

Originally, IPv4 used **classful addressing**.

The 32-bit address space was divided into different classes according to the size of the network and the number of hosts required.

The main reason for having classes was to support networks of **different sizes**.

For example:

- A huge organization may need thousands of hosts.
- A small organization may need only a few hundred hosts.

Therefore, different classes provided different combinations of:

Number of networks $\leftrightarrow$ Number of hosts per network

The five traditional classes are:

Class	First Octet	Default Mask	Network Bits	Host Bits	Networks	Hosts/Network
A	1–126	/8	8	24	126	16,777,214
B	128–191	/16	16	16	16,384	65,534
C	192–223	/24	24	8	2,097,152	254
D	224–239	—	—	—	Multicast	—
E	240–255	—	—	—	Experimental	—

Note: Classful addressing is mostly historical. Modern networks use **CIDR**, but classes are still very important for understanding IPv4 fundamentals and exams.

Class A

Class A was designed for **very large networks**.

0 | Network | Host | Host | Host

- First bit is always 0.
- Network portion = 8 bits.
- Host portion = 24 bits.

Range:

1.0.0.0 – 126.255.255.255

Number of networks

The first bit is fixed as 0.

Therefore:

$$2^7 = 128$$

But 0 and 127 are reserved.

Therefore:

$$128 - 2 = 126 \text{ networks}$$

Hosts per network

There are 24 host bits:

$$2^{24} - 2$$

$$= 16,777,216 - 2$$

$$= 16,777,214 \text{ hosts}$$

We subtract 2 because:

- All host bits 0 → Network address
- All host bits 1 → Broadcast address

Example:

10.0.0.0

can have:

10.0.0.1 → 10.255.255.254

as usable host addresses.

Class B

Class B was designed for **medium-sized networks**.

10 | Network | Network | Host | Host

First two bits:

10

Range:

128.0.0.0 – 191.255.255.255

Network bits available:

$$16 - 2 = 14$$

Therefore:

$$2^{14} = 16,384 \text{ networks}$$

Host bits:

$$16$$

Therefore:

$$2^{16} - 2$$

$$= 65,536 - 2$$

$$= 65,534 \text{ hosts/network}$$

Example:

172.16.0.0

Default mask:

255.255.0.0

or

/16

Class C

Class C was designed for **small networks**.

110 | Network | Network | Network | Host

Range:

192.0.0.0 – 223.255.255.255

The first 3 bits are fixed:

110

Therefore network bits:

$$24 - 3 = 21$$

Number of networks:

$$2^{21} = 2,097,152$$

Host bits:

$$8$$

Usable hosts:

$$2^8 - 2$$

$$= 256 - 2$$

$$= 254$$

Example:

192.168.1.0/24

Usable hosts:

192.168.1.1 – 192.168.1.254

Class D

Class D is used for **multicasting**.

Range:

224.0.0.0 – 239.255.255.255

It does **not** have the normal network/host division.

It is used when data needs to be sent to a **group of devices**.

Example:

224.0.0.1

Class E

Class E was reserved for **experimental/research purposes**.

Range:

240.0.0.0 – 255.255.255.255

It is not used for normal host addressing.

3. Subnetting

Subnetting is the process of dividing one large network into multiple smaller logical networks called **subnets**.

We take some bits from the host portion and use them as subnet bits.

Example

Suppose we have:

192.168.1.0/24

Initially:

Network bits = 24

Host bits = 8

Therefore:

$2^8 - 2 = 254$ hosts

Suppose we borrow **2 host bits**.

Original:

NNNNNNNN.NNNNNNNN.NNNNNNNN.HHHHHHHH

↓

Subnetted:

NNNNNNNN.NNNNNNNN.NNNNNNNN.SSHHHHHH

New prefix:

/26

Number of subnets:

$2^2 = 4$

Hosts per subnet:

$2^6 - 2$

= 62

So:

Subnet Network Address Usable Hosts Broadcast

1	192.168.1.0/26	.1 – .62	.63
2	192.168.1.64/26	.65 – .126	.127
3	192.168.1.128/26	.129 – .190	.191
4	192.168.1.192/26	.193 – .254	.255

Why subnetting?

- Reduces broadcast traffic
- Improves network organization
- Improves address utilization
- Provides logical separation
- Can improve network security

4. Supernetting

Supernetting is the opposite concept of subnetting.

It combines **multiple smaller networks into one larger logical network**.

It is also called **route aggregation** or **route summarization**.

Example

Suppose we have:

192.168.0.0/24

192.168.1.0/24

192.168.2.0/24

192.168.3.0/24

These four /24 networks can be combined into:

192.168.0.0/22

Why?

A /22 contains:

$32 - 22 = 10$ host bits

Total addresses:

$2^{10} = 1024$

Four /24 networks contain:

$4 \times 256 = 1024$ addresses

So:

$4 \times /24 \rightarrow /22$

Subnetting vs Supernetting

Subnetting

Divides a large network

Borrow host bits

Creates smaller networks

Example: /24 → /26

Used for network segmentation

Supernetting

Combines smaller networks

Reduce network bits

Creates larger logical network

Example: /24 + /24 + /24 + /24 → /22

Used for route aggregation

Question 2:

What is the difference between Link state and Distance vector routing? Explain how these two algorithms help in identifying the least cost path with the help of examples.

Solution:

1. What is routing?

Routing is the process of determining the path that packets should follow from a source network to a destination network.

A routing algorithm tries to find the least-cost path.

The cost can be based on:

- Number of hops
- Bandwidth
- Delay
- Reliability
- Administrative cost

Two major routing approaches are:

1. Distance Vector
2. Link State

2. Distance Vector Routing

In Distance Vector routing, each router knows:

Destination + Distance/Cost + Next Hop

A router does not have complete information about the entire network.

It exchanges routing information with its neighboring routers.

The classic algorithm used is the Bellman-Ford algorithm.

The basic equation is:

$$D_x(y) = \min_v [c(x, v) + D_v(y)]$$

Meaning:

The cost from router X to destination Y is the minimum cost of going through one of X's neighbors.

Example

Suppose:

A ----2---- B ----3---- C

\ /

-----6-----

Routes:

A → B = 2

B → C = 3

A → C = 6

A calculates:

Through B:

A → B → C

= 2 + 3

= 5

Direct:

A → C = 6

Therefore:

Minimum = 5

So A chooses:

A → B → C

as the least-cost path.

Characteristics

- Routers exchange information with neighbors.
- Usually sends routing updates periodically.
- Simpler to implement.
- Can converge slowly.
- Can suffer from routing loops and the count-to-infinity problem.

Example protocol:

RIP uses a distance-vector approach.

3. Link State Routing

In Link State routing, each router attempts to build a complete map of the network.

Each router learns:

Which routers are connected

+

Cost of each link

Routers distribute link-state information to other routers.

After building the topology, each router independently calculates the shortest path.

The common algorithm is:

Dijkstra's Shortest Path First (SPF).

Example

Consider:



Possible path from A to D:

Path 1

$A \rightarrow B \rightarrow D$

$= 2 + 3$

$= 5$

Path 2

$A \rightarrow C \rightarrow D$

$= 6 + 1$

$= 7$

Therefore:

$A \rightarrow B \rightarrow D$

is selected because:

$5 < 7$

Dijkstra's algorithm systematically calculates these shortest distances.

Distance Vector vs Link State

Feature	Distance Vector	Link State
Knowledge	Mainly neighbor information	Complete topology
Algorithm	Bellman-Ford	Dijkstra
Information exchanged	Distance/vector information	Link-state information
Updates	Often periodic	Usually event-driven + initial flooding
Convergence	Relatively slower	Generally faster
Complexity	Simpler	More complex
Memory requirement	Lower	Higher
Routing loops	More susceptible	Less susceptible

Feature	Distance Vector	Link State
Example	RIP	OSPF, IS-IS

Easy way to remember

Distance Vector:

"Ask my neighbors how far the destination is."

Link State:

"Build the map and calculate the shortest route myself."

Question 3:

What is congestion in a network? Why do we need to control congestion? Explain the difference between Leaky bucket and Token bucket with examples. Also, explain why do we need to do fragmentation?

Solution:

1. What is congestion?

Congestion occurs when the amount of traffic entering a network is greater than the network's ability to process and forward that traffic.

In simple terms:

Traffic entering

↓

[Router]

↓

Traffic leaving

If:

Incoming traffic > Outgoing capacity

packets start accumulating in buffers.

Eventually, buffers may become full and packets are dropped.

2. Why do we need congestion control?

Congestion can cause:

- Packet loss
- Increased delay
- Increased retransmissions
- Reduced throughput
- Buffer overflow
- Poor network performance

In severe cases, congestion can cause congestion collapse, where increasing traffic results in very little useful data being delivered.

Therefore, congestion control tries to keep traffic within the network's capacity.

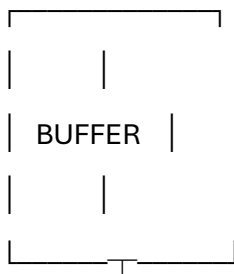
3. Leaky Bucket Algorithm

The Leaky Bucket algorithm is a congestion-control/traffic-shaping technique.

Imagine a bucket with a small hole at the bottom.

Incoming packets

↓



↓

Fixed rate

↓

Network

Packets may arrive at different rates, but they leave at a fixed rate.

Example

Suppose:

Bucket capacity = 10 packets

Output rate = 2 packets/second

Suppose packets arrive:

$t = 0 \rightarrow 6$ packets

$t = 1 \rightarrow 5$ packets

The bucket can store packets temporarily, but they leave at:

2 packets/second

If the bucket becomes full and additional packets arrive, those packets may be discarded.

Key property

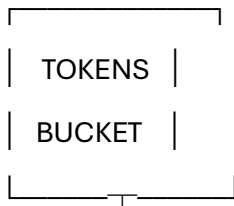
Output rate is relatively constant.

4. Token Bucket Algorithm

In the Token Bucket algorithm, tokens are generated at a fixed rate and stored in a bucket.

A packet can be transmitted only when sufficient tokens are available.

Token generator



Packet transmission



Network

Suppose:

Token generation = 2 tokens/second

Bucket capacity = 10 tokens

If tokens accumulate during an idle period, the sender can later use them to send packets in a burst.

Example

Suppose the system is idle for 3 seconds.

Tokens accumulated:

$$3 \times 2 = 6 \text{ tokens}$$

Now 6 packets arrive simultaneously.

Because 6 tokens are available:

6 packets can be transmitted quickly.

After the tokens are consumed, packets must wait for new tokens.

Leaky Bucket vs Token Bucket

Feature	Leaky Bucket	Token Bucket
Output	Usually fixed/smoothed rate	Allows bursts
Main concept	Packets leak at fixed rate	Tokens authorize transmission
Burst traffic	Not normally allowed	Allowed if tokens accumulated
Overflow	Packets may be discarded	Excess packets may wait/drop depending on implementation
Traffic shaping	Yes	Yes
Flexibility	Less flexible	More flexible

Easy example

Imagine a water bucket:

Leaky Bucket:

Water comes out through a small hole at a fixed rate.

Token Bucket:

You collect tokens while waiting and can spend those tokens to send a burst later.

5. Why do we need Fragmentation?

Different networks can have different MTU (Maximum Transmission Unit) sizes.

The MTU specifies the maximum size of an IP packet that can be transmitted over a particular link.

For example:

Network A

MTU = 1500 bytes

Suppose a packet of:

4000 bytes

needs to cross a network whose MTU is only:

1500 bytes

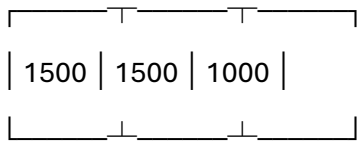
The original packet cannot fit into a single frame.

Therefore, it may need to be divided into smaller pieces called fragments.

Original IP Packet

4000 bytes

↓



Fragment Fragment Fragment

The fragments are transmitted separately and can be reassembled at the destination in IPv4.

Why does MTU differ?

Different technologies have different maximum frame sizes.

For example:

Sender

↓

Network 1

MTU = 4000

↓

Router

↓

Network 2

MTU = 1500

↓

Receiver

The router may encounter a packet that is too large for Network 2.

Therefore, fragmentation may be required in IPv4 if fragmentation is permitted.

Important IPv4 Fragmentation Fields

IPv4 uses fields such as:

Identification

Identifies fragments belonging to the same original IP packet.

Fragment Offset

Tells the receiver where the fragment belongs in the original packet.

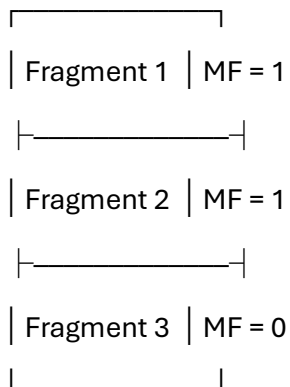
More Fragments (MF) flag

Indicates whether more fragments follow.

For example:

Original Packet

↓



The receiver uses these fields to reconstruct the original packet.